

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №3-4

Выполнил:
Студент группы ИУ5-35Б
Эльсон Э.К.

Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Задание:

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете lab_python_fp. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл field.py)

Необходимо реализовать генератор field. Генератор field последовательно выдает значения ключей словаря. Пример:

```
goods = [
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
    {'title': 'Диван для отдыха', 'color': 'black'}
]
```

field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'

field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}

- В качестве первого аргумента генератор принимает список словарей, дальше через *args генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно None, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно None, то оно пропускается. Если все поля содержат значения None, то пропускается элемент целиком.

Шаблон для реализации генератора:

```
# Пример:
# goods = [
#     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха', 'price': 5300}

def field(items, *args):
    assert len(args) > 0
    # Необходимо реализовать генератор
```

Задача 2 (файл gen_random.py)

Необходимо реализовать генератор gen_random(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона.

Пример:

gen_random(5, 1, 3) должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Шаблон для реализации генератора:

```
# Пример:
# gen_random(5, 1, 3) должен выдать 5 случайных чисел
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1
# Hint: типовая реализация занимает 2 строки
def gen_random(num_count, begin, end):
    pass
    # Необходимо реализовать генератор
```

Задача 3 (файл unique.py)

- Необходимо реализовать итератор `Unique(данные)`, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.
- При реализации необходимо использовать конструкцию `**kwargs`.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Пример:

```
data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

`Unique(data)` будет последовательно возвращать только 1 и 2.

```
data = gen_random(10, 1, 3)
```

`Unique(data)` будет последовательно возвращать только 1, 2 и 3.

```
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

`Unique(data)` будет последовательно возвращать только a, A, b, B.

`Unique(data, ignore_case=True)` будет последовательно возвращать только a, b.

Шаблон для реализации класса-итератора:

```
# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        # Нужно реализовать конструктор
        # В качестве ключевого аргумента, конструктор должен принимать bool-параметр ignore_case,
        # в зависимости от значения которого будут считаться одинаковыми строки в разном регистре
        # Например: ignore_case = True, Абв и АБВ - разные строки
        # ignore_case = False, Абв и АБВ - одинаковые строки, одна из которых удалится
        # По-умолчанию ignore_case = False
        pass

    def __next__(self):
        # Нужно реализовать __next__
        pass

    def __iter__(self):
        return self
```

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа. Необходимо **одной строкой кода** вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`. Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```

Необходимо решить задачу двумя способами:

1. С использованием `lambda`-функции.
2. Без использования `lambda`-функции.

Шаблон реализации:

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = ...
    print(result)

    result_with_lambda = ...
    print(result_with_lambda)
```



Задача 5 (файл print_result.py)

Необходимо реализовать декоратор print_result, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводиться в столбик через знак равенства.

Шаблон реализации:

```
# Здесь должна быть реализация декоратора
```



```
@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()
```

Результат выполнения:

```
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2
```



Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():  
    sleep(5.5)
```



После завершения блока кода в консоль должно вывестись `time: 5.5` (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Задача 7 (файл process_data.py)

- В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.
- В файле [data_light.json](#) содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.
- Предполагается, что функции `f1`, `f2`, `f3` будут реализованы в одну строку. В реализации функции `f4` может быть до 3 строк.
- Функция `f1` должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция `f2` должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова "программист". Для фильтрации используйте функцию `filter`.
- Функция `f3` должна модифицировать каждый элемент массива, добавив строку "с опытом Python" (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию `map`.

- Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист С# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Шаблон реализации:

```
import json
import sys
# Сделаем другие необходимые импорты

path = None

# Необходимо в переменную path сохранить путь к файлу, который был передан при запуске сценария

with open(path) as f:
    data = json.load(f)

# Далее необходимо реализовать все функции по заданию, заменив `raise NotImplemented`
# Предполагается, что функции f1, f2, f3 будут реализованы в одну строку
# В реализации функции f4 может быть до 3 строк

@print_result
def f1(arg):
    raise NotImplemented

@print_result
def f2(arg):
    raise NotImplemented

@print_result
def f3(arg):
    raise NotImplemented

@print_result
def f4(arg):
    raise NotImplemented

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

Листинг кода

cm_timer.py

```
import time
```

```
from contextlib import contextmanager
```

```
class cm_timer_1:
```

```
    def __enter__(self):
```

```
        self.start_time = time.time()
```

```
        return self
```

```
def __exit__(self, exception_type, exception_value, traceback):  
    elapsed_time = time.time() - self.start_time  
    print(f"time: {elapsed_time}")
```

```
@contextmanager
```

```
def cm_timer_2(*args, **kwargs):  
    start_time = time.time()  
  
    try:  
        yield start_time  
    finally:  
        elapsed_time = time.time() - start_time  
        print(f"time: {elapsed_time}")
```

```
if __name__ == "__main__":
```

```
    with cm_timer_1():  
        time.sleep(5.5)
```

```
    with cm_timer_2():  
        time.sleep(5.5)
```

field.py

```
def field(items, *args):
```

```
    assert len(args) > 0
```

```
    result = []
```

```
    if len(args) == 1:
```

```
        result = [item[args[0]] for item in items if args[0] in item.keys()]
```

```

else:
    filtered_items = [item for item in items if any(arg in item.keys() for arg in args)]
    result = [{arg:item[arg] for arg in item.keys() if arg in args} for item in
filtered_items]

return result

if __name__ == "__main__":
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
    ]

    print(field(goods, 'title'))
    print(field(goods, 'title', 'price'))

```

gen_random.py

```

import random

def gen_random(num_count, begin, end):
    nums = [random.randint(begin, end) for i in range(num_count)]

    return nums

if __name__ == "__main__":
    print(gen_random(5, 1, 3))

```

print_result.py

```

def print_result(func):

```



```

def wrapper(*args, **kwargs):
    print(func.__name__)

    result = func(*args, **kwargs)

    if isinstance(result, list):
        print(*result, sep='\n')
    elif isinstance(result, dict):
        print(*[f'{key} = {result[key]}' for key in result.keys()], sep='\n')
    else:
        print(result)

    return result

return wrapper

```

```
@print_result
```

```
def test_1():
    return 1

```

```
@print_result
```

```
def test_2():
    return 'iu5'

```

```
@print_result
```

```
def test_3():
    return {'a': 1, 'b': 2}

```

```
@print_result
```

```

def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!')
    test_1()
    test_2()
    test_3()
    test_4()

```

process_data.py

```

import json

from field import field
from unique import Unique
from gen_random import gen_random
from print_result import print_result
from cm_timer import cm_timer_1

path = "data_light.json"

with open('data_light.json', 'r', encoding='utf-8-sig') as f:
    data = json.load(f)

@print_result
def fl(arg):
    return list(Unique(field(arg, "job-name"), ignore_case=True))

@print_result

```

```
def f2(arg):  
    return list(filter(lambda x: x.startswith("программист") or  
x.startswith("Программист"), arg))
```

```
@print_result
```

```
def f3(arg):  
    return list(map(lambda x: f'{x} с опытом Python', arg))
```

```
@print_result
```

```
def f4(arg):  
    salaries = gen_random(len(arg), 100_000, 200_000)  
  
    return list(map(lambda x: f'{x[0]}, зарплата {x[1]} руб', zip(arg, salaries)))
```

```
if __name__ == '__main__':  
    with cm_timer_1():  
        f4(f3(f2(f1(data))))
```

sort.py

```
if __name__ == '__main__':  
    data = [4, -30, 100, -100, 123, 1, 0, -1, -4]  
  
    result = sorted(data, key=abs, reverse=True)  
    print(result)  
  
    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)  
    print(result_with_lambda)
```

unique.py

```
class Unique:

    def __init__(self, data, **kwargs):
        self.used_elements = set()
        self.data = data
        self.index = 0

        if len(kwargs) == 1 and "ignore_case" in kwargs.keys():
            self.ignore_case = kwargs["ignore_case"]
        else:
            self.ignore_case = False

    def __iter__(self):
        return self

    def __next__(self):
        while True:
            if self.index >= len(self.data):
                raise StopIteration
            else:
                current = self.data[self.index]
                self.index = self.index + 1

                if self.ignore_case and str(current).lower() in [str(i).lower() for i in
self.used_elements]:
                    continue

                if not self.ignore_case and current in self.used_elements:
                    continue
```

```
self.used_elements.add(current)
```

```
return current
```

```
if __name__ == "__main__":
```

```
    data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

```
    print(list(Unique(data)))
```

```
    print(list(Unique(data, ignore_case=True)))
```

Результат выполнения программы:

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\cm_timer.py
```

```
time: 5.5029590129852295
```

```
time: 5.500818490982056
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> 
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\field.py
```

```
['Ковер', 'Диван для отдыха']
```

```
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха', 'price': 5300}]
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> 
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\gen_random.py
```

```
[1, 1, 2, 2, 3]
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> 
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\print_result.py
```

```
!!!!!!!
```

```
test_1
```

```
1
```

```
test_2
```

```
iu5
```

```
test_3
```

```
a = 1
```

```
b = 2
```

```
test_4
```

```
1
```

```
2
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> 
```

Слесарь по ремонту автомобилей 4 разряда-4 разряда
Оператор по добыче нефти и газа 4 разряда-4 разряда
Слесарь-ремонтник 5 разряда-5 разряда
Заместитель директора по учебно-воспитательной работе
Старший контролер ОТК
Экономист на производство
консультант по работе с молодежью
Технический руководитель (в сельском, охотничьем, лесном и рыбном хозяйстве)
заведующая хозяйством
педагог - библиотекарь
Заведующий лабораторией (в прочих отраслях)
старший воспитатель
учитель - технологии
Врач уролог
Электрик участка
Слесарь - ремонтник
Программист/ Junior Developer
Токарь 5-6 разряда
Фрезеровщик 5-6 разряда
преподаватель дисц.ОСНОВЫ ПРОЕКТИРОВАНИЯ ТЕХНОЛОГИЧЕСКИХ ПРОЦЕССОВ
Преподаватель дисц.ЭЛЕКТРОТЕХНИКА
Химик-Лаборант
наладчик технологического оборудования КИПиА
Консультант
Врач ультразвуковой диагностики, ОЛД

f2

Программист
Программист C++/C#/Java
Программист 1С
Программист-разработчик информационных систем
Программист C++
Программист/ Junior Developer
Программист / Senior Developer
Программист/ технический специалист
Программист C#

f3

Программист с опытом Python
Программист C++/C#/Java с опытом Python
Программист 1С с опытом Python
Программист-разработчик информационных систем с опытом Python
Программист C++ с опытом Python
Программист/ Junior Developer с опытом Python
Программист / Senior Developer с опытом Python
Программист/ технический специалист с опытом Python
Программист C# с опытом Python

f4

Программист с опытом Python, зарплата 142128 руб
Программист C++/C#/Java с опытом Python, зарплата 171491 руб
Программист 1С с опытом Python, зарплата 106438 руб
Программист-разработчик информационных систем с опытом Python, зарплата 165626 руб

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\sort.py
[123, 100, -100, -30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 4, -4, 1, -1, 0]
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> █
```

```
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> py .\unique.py
['a', 'A', 'b', 'B']
['a', 'b']
PS C:\Users\elson\OneDrive\Рабочий стол\Лабы ПИКЯП\Lab3-4> █
```